System Programming in C – Homework Exercise 3

Publication date: *Thursday, April 16, 2026*
Due date: *Sunday, May 10, 2026* @ 21:00

General notes:

- A piped sequence of commands is an ordered sequence of Linux commands, each connected to its preceding command using the pipe symbol ('|'). Do not use any other way to combine commands (e.g., *process substitution*, etc.).
- You should only **use commands we saw in class**. For command **`grep`**, you may use the following options (if needed): **`-E -v -c`**. For command **`sed`**, use the **`-r`** option. **Do not use any other option for these two commands!**
- As always, make sure to use relative paths and not absolute paths. Your solution scripts should be executable from any location in the file system. In particular, the only absolute path you should use is `/share/ex_data/ex3/`.

Problem 1:

This problem involves additional parsing tasks for files `nba_roster.txt` and `story.txt`, which you parsed in HW 2. Recall that `story.txt` contains the text of *The Jungle Book* and follows no particular format. File `nba_roster.txt` contains information about the nine tallest players on the 2022 New York Knicks basketball. Every line in the file has information about one player, in the following format:

`<last>``<comma><space>`**`<first>`**`<dash><space>`**`<position>`**`<space>`**`<uniform>`**

- **`<last>`** consists of an uppercase letter followed by any number of lowercase letters (e.g., `Sims` or `Noel`).
- **`<first>`** consists of an uppercase letter with the player's first initial.
- **`<position>`** has one or two upper-case letters, signifying the player's position.
- **`<uniform>`** is the player's number, as written on their shirt, specified by one or two digits.
- `<comma>`, `<space>`, and `<dash>` represent the characters `','`, `' '`, and `'-'`.

Write a piped sequence of commands for each of the tasks specified in 1-7 below. Specify each piped sequence in a separate line in a solution file named `ex3.1.sh`. The file should contain exactly 7 lines, each containing a single command or a single piped sequence of commands.

1. Copy the files `nba_roster.txt` and `story.txt` from directory `/share/ex_data/ex3/` to the current directory.

2. Write a single grep command that creates a file `subset.txt` that contains the lines of `nba_roster.txt` that correspond to players whose first name starts with one of the letters J, C, or M and whose uniform number is at least 10 and less than 40 (i.e., in the range [10-39]).

   Validation: File `subset.txt` should contain the lines from the original file corresponding to Randle, Reddish, and Robinson (in that order).

3. Write a single sed command that creates a file `uniform_nums.txt` that contains the lines of `nba_roster.txt` reformatted as follows:

   ```
   uniform number for <first name initial>. <last name>: <uniform>
   ```

   Validation: File `uniform_nums.txt` should contain the same number of lines as `nba_roster.txt`, and the first two lines should be:

   ```
   uniform number for J. Sims: 45
   uniform number for L. Samanic: 91
   ```

4. Write a piped sequence of commands that creates a file `last_names.txt` that contains a list of players' last names from `nba_roster.txt` that have at least six letters and do not end with `son`. The last names should appear in the same order as in the original file.

   Validation: File `last_names.txt` should contain the following last names: Samanic, Fournier, Randle, Reddish, and Toppin (in this order, one per line).

5. Write a piped sequence of commands that assigns a `bash` variable named `many_punct` to the number of lines in the file `story.txt` that contain at least ten punctuation marks.

   Validation: After applying this command, the variable `many_punct` should hold the value `13` (use `echo` to print it and check).

6. Write a piped sequence of commands that creates a file `first_common_words.txt` that contains a list of words that appear as first words in a line in the story file. Each word in the list should satisfy these requirements:

   - The word starts with an upper-case letter and the remaining characters are lower-case letters.
   - The characters immediately before and after the word are either white spaces or punctuation marks (or the beginning/end of the file).
   - The word appears as a first word in a line of the file `story.txt` at least 30 times and at most 99 times. For a word to be counted it must satisfy the two conditions above and all characters before it in the line must be either white spaces or punctuation marks.
   - Words should be listed in `first_common_words.txt` in alphabetical order.

   Validation: You can find the expected version of the output file in:
   `/share/ex_data/ex3/first_common_words_expected.txt`

7. Write a piped sequence of commands that creates a file `quoted_questions.txt` that contains a list of quoted segments of text that contain question marks and at most 70 characters (including the quotes and question marks). Note the following guidelines:

   - A quoted segment of text starts and ends with a double quote character ("), and it has no other double quote character.
   - The quoted segment can span more than one line in the original file, but you should print it in a single line (convert newline to space).
   - The quoted segment should contain a question mark, and at most 70 characters (including the quotes, spaces, and punctuation marks).

   Validation: You can find the expected version of the output file in:
   `/share/ex_data/ex3/quoted_questions_expected.txt`

Final validation and testing for Problem 1:

1. Verify that file `ex3.1.sh` has exactly 7 lines (use `wc`), and follow the specific validation steps specified for each item in 2-7.
2. Execute the commands in file `ex3.1.sh` as scripts (using `source`) and confirm that the following four files are created in the current directory (use `ls`):
   ```
   first_common_words.txt  last_names.txt
   nba_roster.txt          quoted_questions.txt
   story.txt               subset.txt
   uniform_nums.txt
   ```
3. After executing the script (using `source`) also confirm the value of the bash variable `many_punct` (see validation of Problem 1.5).
4. The script should print nothing to the standard output.
5. Run the script from **various locations** (not just `~/exercises/ex3/`) to ensure that it does not contain unwanted absolute paths.
6. Use the script `/share/ex_data/ex3/test_ex3.1` to test your solution. Execute the following command from directory `~/exercises/ex3/`:

   ```
   ==> /share/ex_data/ex3/test_ex3.1
   ```

   (the script produces a detailed error report to help you debug your code)

Problem 2:

In this problem, we ask you to write two bash scripts for computing the size of a given file and a given directory.

1. Write a bash script named `file-size.sh` that prints the size (in bytes) of a file whose path is stored in a bash variable named `$file`. Follow these guidelines:
   - If variable `$file` contains a valid path to a file (not directory), then the script prints the size of that file in bytes.
   - Otherwise, the script prints this error message:
     `Variable $file does not contain a valid path to a file`

   Implementation notes:
   - Use the appropriate `if` statement to check that variable `$file` contains a valid file path.
   - Extract the file size from the `ls -l` table.

   Execution examples:

```
==> file=/share/ex_data/ex3/dir1/file1
==> source file-size.sh
29

==> file=/share/ex_data/ex3/dir1/the-jungle-book.txt
==> source file-size.sh
273647

==> file=/share/ex_data/ex3/dir1
==> source file-size.sh
Variable $file does not contain a valid path to a file
```

2. Write a bash script named `dir-size.sh` that receives a directory path as an argument in the command line. The script should print the total (sum) size of all files included in that directory subtree (i.e., recursively). The overhead size of directories (4096 bytes) should not be counted here. Follow these guidelines:
   - If the (first) input argument of the script contains a valid path to a file, the script prints the size of that file in bytes (as script `file-size.sh`)
   - If the (first) input argument of the script contains a valid directory (not file), then the script prints the sum of sizes of all files contained within that directory's subtree in the filesystem (see examples below).
   - Otherwise, the script prints this error message:

     `<ARG> is not a valid path`

     Where `<ARG>` is the first argument given to dir-size.sh.

- The exit status of the script should be 0 if a valid path was given as an input argument. Otherwise, the exit status should be 11.
- Only the first argument is considered here. Arguments beyond that should be ignored by the script.

Implementation notes:

- The script should be implemented as a stand-alone executable script. Make sure to have an accurate "`#!`" comment at the top of the file and grant the script executable permissions, to enable this.
- Use the appropriate `if` statements to check that the first input argument contains a valid file path or a valid directory path.
- Use the appropriate call to script `file-size.sh` from Problem 2.1 to get the size of files. You may assume that this script exists in the current directory.
- Use an appropriate recursive call to script `dir-size.sh`. The halting condition of the recursion should be when the input argument corresponds to a file path.
- Use simple arithmetic operations for bash variables to sum sizes.

Execution examples:

```
==> ./dir-size.sh /share/ex_data/ex3/dir1/file1
29
==> echo $?          (← recall that this prints the exit status)
0

==> ./dir-size.sh /share/ex_data/ex3/dir1
1063747
==> echo $?
0

==> ./dir-size.sh /share/ex_data/ex3/dir1/nofile
/share/ex_data/ex3/dir1/nofile is not a valid path
==> echo $?
11

==> ./dir-size.sh /share/ex_data/ex3/dir1/dir2
8682

==> ./dir-size.sh /share/ex_data/ex3/dir1/dir2/dir4
0

==> ./dir-size.sh /share/ex_data/ex3/dir1/dir3
781389

==> ./dir-size.sh /share/ex_data/ex3/dir1/dir3/dir5
781389
```

Validation and testing for Problem 2:

1. Make sure to follow the specific guidelines for each question.
2. Execute your script on the execution examples provided for each question.
3. You should try additional inputs to make sure your scripts provide the required output.
4. Use the testing script `/share/ex_data/ex3/test_ex3.2` to test your solution. This script checks your script using the execution examples we provide here, as well as additional executions. It produces a detailed error report, which could help you debug your code. Execute the following command from directory `~/exercises/ex3/`:

   ```
   ==> /share/ex_data/ex3/test_ex3.2
   ```

Submission Instructions:

1. After you validated and tested your solution, make sure that your `~/exercises/ex3/` directory contains the following scripts:
   - `ex3.1.sh`
   - `file-size.sh`
   - `dir-size.sh`
2. your `~/exercises/ex3/` directory should also contain a `PARTNER` file with the user id of the non-submitting partner. The non-submitting partner should also add a `PARTNER` file containing the user id of the submitting partner.
3. Check your solution by running **`check_ex ex3`**. The script should be executed from the account of the submitting partner, and it may be run from any directory. Clean execution of this script guarantees you 80% of the assignment's grade.
4. Once you are satisfied with your solution, you may submit it by running **`submit_ex ex3`**. The script should be executed from the account of the submitting partner, and may be run from any directory. You may modify your submission any time before the deadline by running **`submit_ex ex3 -o`** from any location.
5. For more information on the submission process, see the **Homework submission instructions** file on the course website.